

Standards für Export von CSV Dateien

Contents

1 Akzeptanzkriterien	3
2 Dateiformat	3
2.1 Trennzeichen (Kommasepariert)	3
2.2 Kopfzeile	3
2.3 Zeichenkodierung	4
2.4 Zeilentrennung	4
2.5 Dateinamen	5
3 Inhalte	5
3.1 Datumsformate	5
3.2 Zeitzone von Zeitstempeln	6
3.3 Zeitdauern	6
3.4 Textfelder (Strings)	7
3.5 Ländercodes	7
3.6 Sprachcodes	8
3.7 Leere Werte	8
3.8 Boolean-Werte	8
3.9 Währungscodes	9
3.10 Dimensionen und Codes	9
3.11 Zahlenformate	10
3.12 Geodaten und Koordinaten	11
3.12.1 WGS 84 (Weltweites Koordinatensystem)	11
3.12.2 CH1903+ / LV95 (Schweizer Koordinatensystem)	11
3.12.3 Dokumentation des Koordinatensystems	12
4 Praktische Beispiele	12
4.1 Vollständige Kundendatei	12
4.2 Artikelstammdaten	12

5 Ablage und Bereitstellung	13
5.1 Ablageorte	13
5.2 Export-Zeitpunkt und Periodizität	13
6 Best Practices und häufige Fehler	14
6.1 Richtig machen	14
6.2 Häufige Fehler vermeiden	14
6.3 Validierung vor Export	14
6.4 Tools zur Validierung	15

Für maschinenlesbare Datenaustausche ist **CSV ist ein Fallback-Format** und sollte nur verwendet werden, wenn modernere Methoden nicht verfügbar sind.

Bevorzugte Alternativen:

- REST APIs (JSON), bevorzugt [OData](#) oder [GraphQL](#)
- Event-basierte Azure-Dienste (ideal für Microsoft Fabric):
 - [Azure Service Bus](#) - Zuverlässiges Cloud Messaging
 - [Azure Event Grid](#) - Event-driven Architektur
 - [Azure Event Hubs](#) - Big Data Streaming (Fabric-integriert)
- Strukturierte Formate (XML mit Schema, Json mit Schema, Parquet, [GeoJSON](#) für Geodaten)
- Database-to-Database Verbindungen

Weitere Informationen zu modernen APIs:

- **OData (Open Data Protocol):** <https://www.odata.org/> - Standard-Protokoll für REST APIs
- **OData Spezifikation:** [OASIS OData v4.0](#) - Offizielle Spezifikation und Dokumentation
- **GraphQL Spezifikation:** <https://spec.graphql.org/> - Offizielle Spezifikation der Query-Sprache
- **GraphQL Learn:** <https://graphql.org/learn/> - Einführung und Tutorials

CSV nur wenn:

- Legacy-Systeme keine APIs unterstützen
- Einmalige Datenmigrationen erforderlich sind
- Einfache Datenübertragung an Endbenutzer nötig ist
- Keine technischen Ressourcen für moderne Integration vorhanden sind

Falls CSV dennoch verwendet werden muss, sind die untenstehenden Standards zwingend einzuhalten, um Datenqualität und Interoperabilität sicherzustellen. Prinzipiell gilt, dass die Verbraucher-Anwendung (Downstream) sich nicht auf jeden Lieferanten-Anwendung (Upstream) separat einstellen muss und ein einheitliches Vorgehen nutzen kann.

1 Akzeptanzkriterien

Die folgenden Akzeptanzkriterien gelten. Sie werden in den weiteren Abschnitten detaillierter beschrieben und mit Referenzen auf offizielle Standards versehen:

1. Zeichenkodierung der exportierten Datei ist UTF-8
2. Dateinamen entspricht Vorgaben
3. Datei hat Kopfzeile
4. Trennzeichen ist Komma
5. Namenskonventionen für Kopfzeile sind eingehalten
6. Zahlenformate entsprechen den Vorgaben
7. Datumsformate, Zeitformate und Dauerformate entsprechen den Vorgaben
8. Zeichenketten (Strings) sind sauber ausgezeichnet
9. Codierungen für Länder, Sprachen oder Währungen entsprechen den Vorgaben
10. Fachliche Codes haben entweder einen separaten Export oder der Text ist in der Datei

2 Dateiformat

2.1 Trennzeichen (Kommasepariert)

CSV-Dateien separieren ihre Spalten prinzipiell mit **Kommas** (,), nicht mit Semikolons oder Tabulatoren.

Beispiel:

```
name,alter,stadt  
"Max Mustermann",25,"Zürich"  
"Anna Beispiel",30,"Bern"
```

Falsch:

```
name;alter;stadt  
"Max Mustermann";25;"Zürich"
```

2.2 Kopfzeile

Jede CSV-Datei muss **zwingend** eine Kopfzeile enthalten, welche die Spaltennamen definiert.

Regeln für Spaltennamen:

- Keine Umlaute (ä, ö, ü → ae, oe, ue / é, è, à, ç → e, e, a, c)
- Keine Leerzeichen oder Sonderzeichen
- Verwendung von snake_case, PascalCase oder camelCase (in dieser Reihenfolge bevorzugt) - siehe [Naming Conventions](#)

- Sprechende Namen verwenden, der Name soll mit der Begrifflichkeit in der Fachabteilung korrelieren (z.B. wie im User Interface der exportierenden Applikation)

Sprache der Spaltennamen:

- **Deutsch:** Bevorzugt für deutschsprachige Anwendungen
- **Französisch:** Für französischsprachige Anwendungen
- **Englisch:** Nur im Notfall, wenn Begrifflichkeit klar oder separat dokumentiert ist

Beispiele:

```
# Gut (Deutsch):
kundenNummer,firmenName,erstellungsDatum
KundenNummer,FirmenName,ErstellungsDatum
kunde_nummer,firmen_name,erstellungs_datum

# Gut (Französisch):
numeroClient,nomEntreprise,dateCreation
NumeroClient,NomEntreprise,DateCreation
numero_client,nom_entreprise,date_creation

# Akzeptabel (Englisch, wenn dokumentiert):
customerNumber,companyName,creationDate
customer_number,company_name,creation_date

# Schlecht:
Kunden-Nummer,Firmen Name,Erstellungs-Datum
Numéro-Client,Nom Entreprise,Date-Création
Customer-Number,Company Name,Creation-Date
```

2.3 Zeichenkodierung

Alle CSV-Dateien müssen zwingend in **UTF-8** ohne BOM (Byte Order Mark) kodiert werden. Dies gewährleistet die korrekte Darstellung von Sonderzeichen und internationalen Schriftzeichen.

Warum UTF-8:

- Universelle Kompatibilität
- Unterstützung aller Unicode-Zeichen
- Standard in modernen Systemen

2.4 Zeilentrennung

CSV-Dateien müssen **Windows-Zeilentrennung** (\r\n) verwenden.

Grund: Maximale Kompatibilität mit Microsoft Excel und anderen Office-Anwendungen.

Technische Details:

- Windows: \r\n (Carriage Return + Line Feed)
- Nicht Unix: \n (nur Line Feed)
- Nicht Mac Classic: \r (nur Carriage Return)

2.5 Dateinamen

Für Stammdaten: Der Dateiname muss immer gleich bleiben. Keine Historisierung durch Dateinamen.

Namenskonvention:

- Kleinbuchstaben verwenden
- Keine Leerzeichen (Unterstriche verwenden)
- Sprechende Namen
- Dateiendung .csv

Beispiele:

```
kunden_stammdaten.csv
artikel_preisliste.csv
mitarbeiter_export.csv
```

3 Inhalte

3.1 Datumsformate

Alle Datums- und Zeitangaben müssen dem **ISO 8601 Standard** entsprechen. Dies gewährleistet eindeutige und maschinell lesbare Zeitangaben.

Format für Datum: YYYY-MM-DD

Format für Datum und Zeit: YYYY-MM-DDTHH:MM:SS oder YYYY-MM-DDTHH:MM:SSZ (UTC)

Alternativ mit expliziter Zeitzone: YYYY-MM-DDTHH:MM:SS+01:00 (für MET/CET) oder YYYY-MM-DDTHH:MM:SS+02:00 (für MESZ/CEST)

Beispiele:

```
datum,zeitstempel,ereignis
"2024-03-15","2024-03-15T14:30:00Z","Bestellung erstellt"
"2024-12-31","2024-12-31T23:59:59Z","Jahresabschluss"
"2024-06-15","2024-06-15T14:30:00+02:00","Sommerzeit-Ereignis"
"2024-01-15","2024-01-15T14:30:00+01:00","Winterzeit-Ereignis"
```

Wichtige Regeln:

- Datum muss gültig sein (keine unmöglichen Daten wie 31. Februar)
- Leeres Feld wenn kein Datum bekannt ist
- Zeitbereich muss mit dem Zielsystem kompatibel sein

3.2 Zeitzone von Zeitstempeln

Zeitstempel müssen entweder in **UTC** angegeben oder die Zeitzone direkt im Zeitstempel kodiert werden.

Empfohlen (UTC):

```
ereignis_zeit,beschreibung
"2024-10-24T10:15:00Z","Meeting gestartet"
"2024-10-24T11:45:00Z","Meeting beendet"
```

Alternativ mit Zeitzone im Zeitstempel:

```
ereignis_zeit,beschreibung
"2024-10-24T12:15:00+02:00","Lokales Meeting (Sommerzeit)"
"2024-12-24T12:15:00+01:00","Lokales Meeting (Winterzeit)"
```

3.3 Zeitdauern

Zeitdauern müssen dem **ISO 8601 Duration Standard** entsprechen und mit P beginnen.

Format: P[n]Y[n]M[n]DT[n]H[n]M[n]S

Erklärung:

- P = Period (Dauer-Kennzeichnung)
- Y = Jahre, M = Monate, D = Tage
- T = Time-Trenner (trennt Datum von Uhrzeit)
- H = Stunden, M = Minuten, S = Sekunden

Beispiele:

```
aufgabe,geplante_dauer,tatsaechliche_dauer
"Meeting Vorbereitung","PT30M","PT45M"
"Projektphase 1","P3M","P3M2W"
"Pause","PT15M","PT20M"
"Gesamtprojekt","P1Y6M","P1Y8M3D"
```

Häufige Formate:

- PT30M = 30 Minuten
- PT2H = 2 Stunden
- PT2H30M = 2 Stunden 30 Minuten

- P1D = 1 Tag
- P1W = 1 Woche
- P1M = 1 Monat
- P1Y = 1 Jahr
- P1Y2M3DT4H5M6S = 1 Jahr, 2 Monate, 3 Tage, 4 Stunden, 5 Minuten, 6 Sekunden

Alternative für einfache Fälle: Für reine Stunden/Minuten kann auch das Format HH:MM:SS verwendet werden:

```
aufgabe,dauer_hhmmss
"Kurzes Meeting","00:30:00"
"Langes Meeting","02:15:00"
"Workshop","08:00:00"
```

3.4 Textfelder (Strings)

Alle Textfelder müssen mit **doppelten Anführungszeichen** umschlossen werden. Auch Datumsfelder sollten konservativ mit doppelten Anführungszeichen umschlossen werden. Umbrüche innerhalb von Zeichenketten werden so automatisch korrekt interpretiert.

Escaping von Anführungszeichen: Anführungszeichen innerhalb eines Strings werden durch Verdopplung escaped.

Beispiele:

```
name,beschreibung,kommentar
"Max Mustermann","Kunde seit 2020","Sehr zufrieden"
"Anna Beispiel","Neue Kundin","Sagte: ""Sehr gut!"""
```

Referenz: Gemäss [RFC 4180](#)

3.5 Ländercodes

Für Ländercodes muss der [ISO 3166-1](#) Standard verwendet werden.

Empfohlen: 3-Zeichen Alpha-3 Code (ISO 3166-1 alpha-3)

Alternativ: 2-Zeichen Alpha-2 Code (ISO 3166-1 alpha-2)

Beispiele:

```
# 3-Zeichen Codes (empfohlen):
kunde,land_code,land_name
"Mustermann AG","CHE","Schweiz"
"Example Corp","DEU","Deutschland"
"Société Exemple","FRA","Frankreich"
```

```
# 2-Zeichen Codes (alternativ):
kunde, land_code, land_name
"Mustermann AG", "CH", "Schweiz"
"Example Corp", "DE", "Deutschland"
```

Länder, die nicht mehr durch die ISO Gremien kodiert werden (z.B. da sie in den Neunziger Jahren nur temporär gültig waren) gelten als mangelnde Datenqualität und werden entsprechend ausgezeichnet.

3.6 Sprachcodes

Sprachcodes müssen dem [ISO 639](#) Standard entsprechen.

Empfohlen: 2-Zeichen Sprachcode ([ISO 639-1](#))

Erweitert: Mit Ländercode ([RFC 5646 / BCP 47](#))

Beispiele:

```
# Einfache Sprachcodes:
kunde, sprache, bevorzugte_kommunikation
"Mustermann AG", "de", "Deutsch"
"Example Corp", "en", "Englisch"
"Société Exemple", "fr", "Französisch"

# Mit Ländercode:
kunde, sprache_region, bemerkung
"Schweizer Firma", "de-CH", "Schweizerdeutsch"
"Österreich GmbH", "de-AT", "Österreichisches Deutsch"
"Deutsche AG", "de-DE", "Hochdeutsch"
```

3.7 Leere Werte

Leere oder unbekannte Werte werden als **komplett leere Felder** dargestellt (nicht als "NULL", "N/A" oder ähnliches).

Beispiel:

```
name, alter, email, telefon
"Max Mustermann", 25, "max@example.com", "+41 44 123 45 67"
"Anna Beispiel", , "anna@example.com",
"Peter Test", 30, , "+41 31 987 65 43"
```

3.8 Boolean-Werte

Boolean-Werte (Wahrheitswerte) werden **ohne Anführungszeichen** angegeben.

Erlaubte Formate:

- true / false (empfohlen)
- 1 / 0 (alternativ)

Beispiele:

```
kunde,aktiv,premium_kunde,newsletter_aktiv  
"Mustermann AG",true,false,1  
"Example Corp",false,true,0  
"Test GmbH",true,true,1
```

Nicht verwenden:

- "ja" / "nein"
- "TRUE" / "FALSE"
- "wahr" / "falsch"

3.9 Währungscodes

Währungscodes müssen dem [ISO 4217 Standard](#) entsprechen.

Format: 3-stelliger alphabetischer Code

Beispiele:

```
produkt,preis,waehrung,land  
"Laptop",1299.00,"CHF","Schweiz"  
"Software",299.99,"EUR","Deutschland"  
"Service",150.00,"USD","USA"
```

Häufige Währungscodes:

- CHF = Schweizer Franken
- EUR = Euro
- USD = US-Dollar
- GBP = Britisches Pfund

3.10 Dimensionen und Codes

Bei Verwendung von Codes (Schlüsseln) sollten sowohl der **Code** als auch die **sprechende Bezeichnung** enthalten sein, ausser wenn eine separate Code-Tabelle verfügbar ist.

Empfohlen - Code und Bezeichnung:

```
mitarbeiter,gruppe_code,gruppe_name,kst_code,kst_name  
"Max Mustermann","MG001","Geschäftsleitung","KS100","IT-Abteilung"  
"Anna Beispiel","MG002","Sachbearbeitung","KS200","Marketing"  
"Peter Test","MG003","Entwicklung","KS100","IT-Abteilung"
```

Alternativ - nur Code (wenn separate Referenztafel vorhanden):

```
mitarbeiter,mitarbeitergruppe_code,kostenstelle_code
"Max Mustermann","MG001","KS100"
"Anna Beispiel","MG002","KS200"
"Peter Test","MG003","KS100"
```

Beispiele für Dimensionen:

- Mitarbeitergruppen (mitarbeitergruppe_code, mitarbeitergruppe_name)
- Kostenstellen (kostenstelle_code, kostenstelle_name)
- Produktkategorien (kategorie_code, kategorie_name)
- Organisationseinheiten (abteilung_code, abteilung_name)
- Status-Codes (status_code, status_bezeichnung)

Vorteile:

- **Lesbarkeit:** Daten sind ohne Nachschlagen verständlich
- **Vollständigkeit:** Alle Informationen in einer Datei
- **Robustheit:** Funktioniert auch ohne separate Referenztabellen

3.11 Zahlenformate

Für alle numerischen Werte in CSV-Dateien gelten folgende zwingenden Regeln:

Dezimaltrennzeichen: Zwingend **Punkt** (.) verwenden, niemals Komma

Tausendertrennzeichen: **Keine** Tausendertrennzeichen verwenden (keine Leerzeichen, Apostrophe oder Punkte)

Führende Nullen: Bei Dezimalzahlen unter 1 muss eine führende Null stehen. Ansonsten sollen keine führenden Nullen (gerade bei Schlüsselwerten) ausgegeben werden. Beispiel 00032058 wäre eine schlechte Ausgabe einer Mitarbeiternummer.

Negative Zahlen: Minuszeichen direkt vor der Zahl (ohne Leerzeichen)

Beispiele:

```
betrag,menge,prozent,negativ_wert
1250.50,1000,15.75,-125.00
0.99,5,0.5,-0.25
1000000,999999,100.0,-1000000
```

Falsch (deutsche/europäische Formatierung):

```
# Falsch - nicht verwenden:
betrag,menge,prozent,negativ_wert
"1.250,50","1'000","15,75","125.00-"
"1 250.50",1.000.000,"0,99","0,25-"
125-,50-,10,5-
```

3.12 Geodaten und Koordinaten

Für die Darstellung von Geodaten in CSV-Dateien gelten spezifische Standards abhängig vom verwendeten Koordinatensystem.

3.12.1 WGS 84 (Weltweites Koordinatensystem)

Für Geodaten im weltweiten Referenzsystem **WGS 84** gelten folgende Regeln:

Koordinatenformat:

- Koordinaten müssen in **Dezimalgrad** (Decimal Degrees) angegeben werden
- **Reihenfolge:** Latitude (Breitengrad) vor Longitude (Längengrad)

Wertebereich:

- **Latitude:** Numerischer Wert zwischen -90.0 und 90.0
- **Longitude:** Numerischer Wert zwischen -180.0 und 180.0

Spaltennamen:

- latitude oder lat für Breitengrad
- longitude oder lon für Längengrad

Beispiel:

```
fid,name,street,latitude,longitude
999,"Kontrollgebäude","Zentralstrasse 49",47.136960,7.247312
1001,"Rathaus","Mühlebrücke 5",47.137500,7.245800
1002,"Hauptbahnhof","Bahnhofplatz 1",47.135000,7.244500
```

3.12.2 CH1903+ / LV95 (Schweizer Koordinatensystem)

Für Geodaten im Schweizer Referenzsystem **CH1903+ / LV95** (Landesvermessung 1995) gelten folgende Regeln:

Koordinatenformat:

- Koordinaten müssen in **Metern** angegeben werden
- **Reihenfolge:** E (Easting/Rechtswert) vor N (Northing/Hochwert)

Referenzpunkt:

- Ursprung: Alter Astronomischer Observatorium Bern
- E = 2'600'000.000 m
- N = 1'200'000.000 m

Spaltennamen:

- e_lv95 oder easting für Rechtswert (Ost-Koordinate)
- n_lv95 oder northing für Hochwert (Nord-Koordinate)

Zahlenformat:

- Dezimaltrennzeichen: **Punkt** (.)
- Tausendertrennzeichen: **Keine** (nicht ' oder Leerzeichen verwenden)

Beispiel:

```
fid,name,street,e_lv95,n_lv95
999,"Kontrollgebäude","Zentralstrasse 49",2585486.22,1220682.34
1001,"Rathaus","Mühlebrücke 5",2585450.50,1220700.80
1002,"Hauptbahnhof","Bahnhofplatz 1",2585380.00,1220650.00
```

3.12.3 Dokumentation des Koordinatensystems

Das verwendete Koordinatensystem **muss** im Dateinamen dokumentiert werden:

```
gebaeude_wgs84.csv
gebaeude_lv95.csv
standorte_wgs84.csv
```

Weitere Koordinatensysteme:

Falls andere Koordinatensysteme verwendet werden, muss dies ebenfalls dokumentiert werden mit:

- EPSG-Code (z.B. EPSG:4326 für WGS84, EPSG:2056 für CH1903+/LV95)
- Im Dateinamen

4 Praktische Beispiele

4.1 Vollständige Kundendatei

```
kunden_nummer,firmen_name,ansprechperson,email,telefon,land_code,sprache,
  ↳ erstellungs_datum,aktiv
1001,"Mustermann AG","Max Mustermann","max.mustermann@example.com","+41
  ↳ 44 123 45 67","CHE","de-CH","2024-01-15T09:30:00Z",true
1002,"Example Corporation","Jane Smith","jane.smith@example.com","+1 555
  ↳ 123 4567","USA","en","2024-02-20T14:15:00Z",true
1003,"Société Exemple","Pierre Dupont","pierre.dupont@exemple.fr","+33 1
  ↳ 23 45 67 89","FRA","fr","2024-03-10T11:45:00Z",false
```

4.2 Artikelstammdaten

```
artikel_nummer,bezeichnung,kategorie,preis_chf,lagerbestand,lieferant,
  ↳ verfuegbar_ab,bemerkung
```

```
"ART-001","Laptop Standard","Elektronik",1299.00,25,"Tech Supplier AG"
  ↳ ", "2024-01-01T00:00:00Z", "Standardkonfiguration"
"ART-002","Bürostuhl ergonomisch","Möbel",450.50,12,"Office Solutions"
  ↳ ", "2024-02-15T00:00:00Z", "Höhenverstellbar"
"ART-003","Software Lizenz Pro","Software",299.99,0,"Software AG",,"""
  ↳ Professional"" Version mit Vollsupport"
```

5 Ablage und Bereitstellung

5.1 Ablageorte

Die Ablage von CSV-Dateien ist systemabhängig und muss an die vorhandene IT-Infrastruktur angepasst werden.

Mögliche Ablageorte:

On-Premise:

- Shared Network Drives (\\server\share\csv_exports)
- FTP/SFTP Server
- Lokale Dateiserver

Cloud-basiert:

- [Azure Blob Storage](#) - Objektspeicher für große Datenmengen
- [Azure File Share](#) - Cloud-basierte Dateifreigaben (SMB)
- [SharePoint Document Libraries](#) - Dokumentenverwaltung und Zusammenarbeit
- [OneDrive for Business](#) (nur für kleine Dateien)

Sicherheitsanforderungen:

- Zugriffsberechtigung nur für autorisierte Benutzer
- Audit-Log für Dateizugriffe mit vertraulichen Daten
- Automatische Löschung nach definierter Aufbewahrungszeit

5.2 Export-Zeitpunkt und Periodizität

Der Zeitpunkt und die Häufigkeit des CSV-Exports müssen klar definiert und dokumentiert werden.

Zu definierende Aspekte:

Zeitpunkt:

- **Täglich:** Um 06:00 Uhr (vor Geschäftsbeginn). Falls nicht UTC als Zeitzone verwendet wird, muss das Verhalten Sommerzeit/Winterzeit unbedingt detailliert geklärt sein.
- **Wöchentlich:** Montags um 05:00 Uhr

- **Monatlich:** Am 1. des Monats um 02:00 Uhr
- **Ad-hoc:** Auf Anfrage durch autorisierte Benutzer (in der Regel immer zu vermeiden)

Datenstand:

- **Snapshot-Zeit:** Welcher Datenstand wird exportiert?
- **Cut-off:** Bis wann sind Änderungen im Export enthalten?
- **Zeitzone:** In welcher Zeitzone erfolgt der Export?

Benachrichtigung:

- **Erfolg:** E-Mail an Datenempfänger
- **Fehler:** Sofortige Benachrichtigung an IT-Support
- **Log-Dateien:** Detaillierte Protokollierung für Nachvollziehbarkeit

Beispiel-Dokumentation:

```
Export: Kundenstammdaten
Zeitpunkt: Täglich 06:00 Uhr MEZ
Datenstand: Bis 23:59 Uhr des Vortags
Ablageort: `\\fileserver\exports\kunden\kunden_YYYYMMDD.csv`
Aufbewahrung: 30 Tage
Benachrichtigung: datenverwaltung@firma.ch
```

6 Best Practices und häufige Fehler

6.1 Richtig machen

- **Konsistente Formatierung:** Alle Datums- und Zeitangaben im gleichen Format
- **Sprechende Spaltennamen:** kunden_nummer statt kn
- **Vollständige Kopfzeile:** Jede Spalte muss benannt sein
- **UTF-8 Kodierung:** Immer ohne BOM verwenden
- **Validation:** Daten vor Export validieren

6.2 Häufige Fehler vermeiden

- **Semikolons als Trennzeichen:** In CSV immer Kommas verwenden
- **Inkonsistente Datumsformate:** Nicht 15.03.2024 und 2024-03-15 mischen
- **Fehlende Anführungszeichen:** Bei Texten mit Kommas oder Sonderzeichen
- **Lokale Zahlenformate:** 1.250,50 statt 1250.50
- **Umlaute in Spaltennamen:** grösse → groesse

6.3 Validierung vor Export

Vor dem Export sollten folgende Punkte geprüft werden:

1. **Datentypen:** Sind alle Spalten korrekt formatiert?
2. **Vollständigkeit:** Sind alle Pflichtfelder gefüllt?
3. **Eindeutigkeit:** Sind Primärschlüssel eindeutig?
4. **Gültigkeitsbereiche:** Liegen Werte in erlaubten Bereichen?
5. **Zeichenkodierung:** Ist die Datei korrekt als UTF-8 gespeichert?

6.4 Tools zur Validierung

Empfohlene Tools:

- **csvlint:** Online-Validierung von CSV-Dateien
- **Excel/LibreOffice:** Export-Funktionen mit UTF-8 Option
- **Texteditor:** Zur Überprüfung der Zeichenkodierung

UTF-8 Kodierung prüfen (PowerShell):

```
# Kodierung einer Datei prüfen
Get-Content "datei.csv" -Encoding Byte -TotalCount 3 | ForEach-Object {
    ↪ "{0:X2}" -f $_ }

# Wenn die ersten 3 Bytes "EF BB BF" sind = UTF-8 mit BOM (nicht
    ↪ erwünscht)
# Wenn andere Werte = UTF-8 ohne BOM (korrekt) oder andere Kodierung
```

Beispiel-Validierung in Python:

```
import csv
import datetime

def validate_csv(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        reader = csv.DictReader(file)
        for row_num, row in enumerate(reader, 1):
            # Validierung der Datumsfelder
            if 'datum' in row and row['datum']:
                try:
                    datetime.datetime.fromisoformat(row['datum'].replace(
                        ↪ 'Z', '+00:00'))
                except ValueError:
                    print(f"Zeile_{row_num}:_Ungültiges_Datum_{row['datum']}")
```